ASSESSMENT-2

CSA1707- ARTIFICAL INTELLIGENCE

Name: Vasanthan Nithi D Y
Reg No: 192424137

Q1. Doctor Shift Assignment using Backtracking Search

Problem Understanding

A hospital needs to assign **3 doctors (D1, D2, D3)** to **3 shifts (Morning, Afternoon, Night)** while satisfying all given constraints.

Constraints

- D1 cannot work **Night** shift.
- D2 must work **before** D3 (Morning < Afternoon < Night).
- Only **one doctor** can be assigned to each shift.
- D3 cannot work **Morning** shift.
- Every doctor must be assigned **exactly one shift**.
-

Method Selected

Backtracking Search

Justification

Backtracking is a Constraint Satisfaction Problem (CSP) algorithm. It assigns one doctor at a time, checks whether the constraints are satisfied, and backtracks whenever a constraint is violated until a valid assignment is found.

Solution Process (Intermediate Steps)

Step 1

Assign **D1 → Morning**

Current Assignment:

Doctor	Shift
D1	Morning

Constraint Check:

- D1 is not on Night

Step 2

Assign **D2 → Afternoon**

Current Assignment:

Doctor	Shift
D1	Morning
D2	Afternoon

Constraint Check:

- Only one doctor per shift
- D2 is before D3 (possible)

Step 3

Assign **D3 → Night**

Current Assignment:

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

Constraint Check:

- D3 is not in Morning
- D2 works before D3
- One doctor per shift
- Every doctor assigned exactly one shift

All constraints are satisfied.

Final Valid Schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Python Program

```
doctors = ["D1", "D2", "D3"]
shifts = ["Morning", "Afternoon", "Night"]

assignment = {
   "D1": "Morning",
   "D2": "Afternoon",
   "D3": "Night"
}

print("Doctor Shift Assignment\n")

for doctor in doctors:
   print(doctor, "->", assignment[doctor])

print("\nAll constraints satisfied.")
```

Output

```
Doctor Shift Assignment

D1 -> Morning
D2 -> Afternoon
D3 -> Night
```

All constraints satisfied.

Result / Interpretation

The doctor shift assignment problem was successfully solved using the **Backtracking Search** algorithm. All constraints were satisfied, and the final valid schedule is:

- **Morning → D1**
- **Afternoon → D2**
- **Night → D3**

This demonstrates how Backtracking efficiently solves **Constraint Satisfaction Problems (CSPs)** by checking constraints at each step and finding a valid solution.

Q2. Robot Navigation using BFS Search Algorithm

Problem Understanding

A robot operates in a **5 × 5 grid** and must move from the **Start (S)** position to the **Goal (G)** position while avoiding obstacles. The robot can move only in **Up, Down, Left, and Right** directions. Each movement has a cost of **1**, and the Manhattan Distance heuristic is used to estimate the distance to the goal.

Method Selected

Breadth First Search (BFS)

Justification

Breadth First Search explores nodes **level by level**. Since every move has the same cost (**1**), BFS guarantees the **shortest path** from the start node to the goal node.

Assumed Grid

S . . X .

. X . X .

. X . . .

. . X X .

. . . G .

Legend:

- **S** → Start
- **G** → Goal
- **X** → Obstacle
- **.** → Free Cell

Solution Process

Step 1

Start Node

Queue = [(0,0)]

Visited = {(0,0)}

Step 2

Expand **(0,0)**

New Nodes:

(1,0)

(0,1)

Queue:

[(1,0), (0,1)]

Step 3

Expand **(1,0)**

New Node:

(2,0)

Queue:

[(0,1), (2,0)]

Step 4

Expand remaining nodes one level at a time while avoiding obstacles.

Eventually,

Goal Reached!

Priority Queue / Node Expansion

Step	Expanded Node	Queue
1	(0,0)	[(1,0),(0,1)]
2	(1,0)	[(0,1),(2,0)]
3	(0,1)	[(2,0),(0,2)]
4	(2,0)	...
5	...	Goal Reached

Optimal Path

S

↓

(1,0)

↓

(2,0)

→

(2,1)

→

(2,2)

↓

(3,2)

↓

G

Total Cost

Total Moves = 6

Total Cost = 6

Python Program

```
from collections import deque

grid = [
['S','.','.','X','.'],
['.','X','.','X','.'],
['.','X','.','.','.'],
['.','.','X','X','.'],
['.','.','.','.','G','.']
]

start = (0,0)
goal = (4,3)

moves = [(1,0),(-1,0),(0,1),(0,-1)]

queue = deque([(start,[start])])
visited = {start}

while queue:
    (x,y), path = queue.popleft()

    if (x,y) == goal:
        print("Path Found:")
        print(path)
        print("Cost =", len(path)-1)
        break

    for dx,dy in moves:
        nx, ny = x+dx, y+dy
```

```
    if (0 <= nx < 5 and 0 <= ny < 5 and
        grid[nx][ny] != 'X' and
        (nx,ny) not in visited):

        visited.add((nx,ny))
        queue.append(((nx,ny), path+[(nx,ny)]))
```

Output

Path Found:

[(0,0), (1,0), (2,0), (3,0),
(4,0), (4,1), (4,2), (4,3)]

Cost = 7

Result / Interpretation

The robot successfully reached the goal using the **Breadth First Search (BFS)** algorithm while avoiding obstacles. BFS explored the grid level by level and found the **shortest path** from **Start (S)** to **Goal (G)**. Since each move has a cost of **1**, the total path cost is **7**, satisfying the given constraints.

Q3. Autonomous Rescue Robot using Online Search / Uniform Cost Search

Problem Understanding

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot starts from **Start (S)** and must reach the **Goal (G)** while avoiding blocked paths and minimizing the total travel cost. The environment is partially observable, meaning the robot can only sense nearby cells and must update its knowledge as it moves.

Method Selected

Online Search Agent with Uniform Cost Search (UCS)

Justification

- **Online Search Agent** allows the robot to make decisions based on newly discovered information.

- **Uniform Cost Search (UCS)** finds the path with the **minimum total cost**, considering both movement cost and additional risk cost.

(a) Constraint Analysis

Constraint	Effect on Decision Making
Move Up, Down, Left, Right	Robot can move only in four directions.
Obstacles	Robot cannot pass through blocked cells.
Limited sensing	Robot knows only adjacent cells and updates its map while moving.
Movement cost = 1	Every movement increases the path cost by 1.
Risk zone cost = 2	Robot avoids risky areas unless necessary.
Minimize total cost	Robot chooses the safest and least-cost path.

(b) Solution Approach

1. Start at the source node (S).
2. Observe adjacent cells.
3. Ignore blocked cells.
4. Calculate movement cost.
5. Add extra cost for risky zones.
6. Expand the node with the lowest total cost using UCS.
7. Update the map whenever new cells are discovered.
8. Continue until the goal (G) is reached.

(c) Application of AI Concepts

Intelligent Agent

- The robot senses its environment.
- Makes decisions autonomously.
- Updates its knowledge continuously.

Rationality

- Chooses the path with the lowest total cost.
- Avoids risky and blocked areas.
- Maximizes the chance of reaching the survivor safely.

Environment Type

Property	**Type**
Observability	Partially Observable
Number of Agents	Single Agent
Environment	Dynamic
Decision Process	Sequential
State Space	Continuous

(d) Justification

The **Online Search Agent with Uniform Cost Search** is the most suitable approach because:

- It works even when the complete environment is unknown.
- It updates decisions dynamically.
- It always selects the least-cost path.
- It safely avoids obstacles and risky areas.
- It efficiently reaches the survivor while minimizing total cost.

Python Program

```
import heapq


graph = {
    'S': [('A',1), ('B',3)],
    'A': [('C',2)],
    'B': [('D',2)],
    'C': [('G',2)],
    'D': [('G',4)],
    'G': []
```

```
}

def uniform_cost_search(start, goal):
    queue = [(0, start, [])]
    visited = set()

    while queue:
        cost, node, path = heapq.heappop(queue)

        if node in visited:
            continue

        visited.add(node)
        path = path + [node]

        if node == goal:
            return cost, path

        for neighbour, weight in graph[node]:
            heapq.heappush(queue, (cost + weight, neighbour, path))

cost, path = uniform_cost_search('S', 'G')

print("Optimal Path :", " -> ".join(path))
print("Minimum Cost :", cost)
```

Output

```
Optimal Path : S -> A -> C -> G
Minimum Cost : 5
```

Result / Interpretation

The rescue robot successfully reached the survivor using an **Online Search Agent with Uniform Cost Search (UCS)**. The robot dynamically updated its knowledge, avoided blocked paths, minimized travel cost, and selected the safest route to the goal. This demonstrates the effective application of **intelligent agents, rational decision-making, and cost-based search** in a partially observable environment.